

Physical Attacks: Technical Application

Attack Vectors	Example	Countermeasures	Cost	Usability	Security
Physical tampering	Removing screws, open enclosure	Use a metal mesh connected to a tamper detecting controller.	Medium, as it would require mid-level-sophisticated hw/sw resource	Low/Medium, as it may slightly hinder authorized maintenance	High, as it prevents simple extraction while providing detection
Chip decapping for SoC extraction	Removing chip and read internal flash or extract keys	Keeping keys in a dedicated secure element that never exposes raw keys. Also, using encrypted firmware images signed by device vendor itself is encouraged.	High, as it requires well setup and highly sophisticated hw/sw, human resources as well	Low, as it is transparent to users	High, as it increases complexity.
Port abuse/JTAG access	Access exposed debug headers to dump memory and/or patch firmware	Seal off debug in production, require physical key for debug for authorized service only.	Low, as it requires very low effort and sophistication to implement	Medium, as it restricts field debugging	High, as it blocks a common and low cost attack path
Wireless physical promity attacks	Sniff or relay commands to control device	Use authenticated encrypted links, re-pairing with physical confirmation (like a button or something).	Low, as this implementation can be handled softly (software wise)	Low, as it may require user action for pairing	High, as it protects over-the-air controls
Fault injection	Inject voltage glitch to skip secure boot signature checks or enable bootloader	Implement secure boot that verifies signatures in hardware backed root or trust.	Medium, as most of the implementation relies on software/sw part of the measure	Medium, as it may require robust power design	High, as it prevents bypassing of firmware validation